Digital-Twin & Synthetic-Data Partner/Vendor Integration Framework

Implementation Solution Guide

Version 1.0.0 · 2026-07-06

Contents

Executive Summary 4

Solution Overview 4

What this is . . . 4

How it works . . . 5

Target model . . . 5

The deployment model — Assess → Design → Generate/Build → Validate → Expand . . . 6

Benefits . . . 6

Limitations . . . 6

Requirements 7

1. A candidate vendor set with obtainable evaluation artifacts . . . 7

2. A declared target-model interface contract . . . 7

3. Two isolated adapter boundaries (data-in, data-out) . . . 8

4. Evaluation criteria with agreed weights and thresholds . . . 8

5. A privacy/validation reference and disclosure-risk threshold for vendor output . . . 8

6. A documented exit/rollback expectation per vendor . . . 9

Global Requirements — Synthetic Data & AI Test-Enablement Solutions 9

1. A Python data/engineering environment . . . 9

2. A target clinical data model / standard . . . 10

3. An isolated non-production test environment . . . 10

4. Privacy, governance & compliance stakeholders and policy . . . 10

5. AI/QA use cases and a pilot scope . . . 10

6. Reference/real-data access for fidelity (read-only, minimized) . . . 11

Implementation 11

Stage 0 — Preconditions . . . 11

Stage 1 — Assess (evaluate candidate vendors) . . . 12

Stage 2 — Design (contract + adapters + exit) . . . 12

Stage 3 — Generate/Build (run the pilot integration) . . . 12

Stage 4 — Validate (the pass/fail gates) . . . 13

Stage 5 — Expand . . . 13

Testing & Validation 14

Test 1 — Contract conformance (Gate B) . . . 14

Test 2 — Adapter conformance / round-trip (Gate C) . . . 14

Test 3 — Fidelity & coverage (Gate D) . . . 14

Test 4 — Privacy / disclosure-risk validation (Gate E) . . . 15

Test 5 — Portability / lock-in (Gate F) . . . 15

Gate summary & rollback . . . 16
Troubleshooting 16
Gate A — no candidate clears the selection threshold . . . 16
Scorecard weights don't sum to 1.0 . . . 16
Contract file invalid (before any data) . . . 16
Gate B FAIL — missing fields or broken references . . . 17
Gate C FAIL — vendor-native shape leaking into the harness . . . 17
Gate D — fidelity below tolerance or coverage gaps . . . 17
Gate E FAIL — exact matches or below-k records . . . 17
Gate F FAIL — low portability score or export won't re-load . . . 18
A gate passes but the vendor still feels "sticky" across programs . . . 18
Version drift after a vendor upgrade . . . 18
Best Practices 18
Treat the target model as the contract, not the vendor . . . 18
Isolate vendor code to exactly two adapters . . . 19
Score before you select, and keep the evidence . . . 19
Design the exit on day one . . . 19
Hold partner output to your own privacy bar . . . 19
Keep the safety and boundary discipline . . . 19
Govern the portfolio, not just the pilot . . . 20
Deliverables Summary 20
Documentation . . . 20
The 5 Python tools . . . 20
Appendix A — PowerShell Scripts 22

Executive Summary

Outcome. Partner digital-twin and synthetic-data capabilities — evaluated, integrated, and kept **portable** — so they are leveraged consistently across programs without vendor lock-in. Instead of one-off, hand-wired integrations that quietly harden into dependencies, each partner capability becomes a **swappable component behind a stable, testable contract**.

Approach. The framework standardizes the boundary between your environment and any external vendor and applies the house model **Assess** → **Design** → **Generate/Build** → **Validate** → **Expand**, pilot-first:

• **Assess** — score candidate vendors on agreed weighted criteria (fidelity, privacy posture,

interoperability, portability, support, cost) with a repeatable scorecard; select on evidence.

• **Design** — declare a **target-model interface contract** (FHIR / OMOP / native) and isolate

all vendor-specific code to exactly two thin adapters (data-in, data-out); document the exit path up front.

• **Generate/Build** — run the vendor to produce synthetic output (or execute a twin/scenario)

into an **isolated non-production** environment, shaped to the target-model contract.

• **Validate** — five explicit pass/fail gates: contract conformance, adapter conformance,

fidelity/coverage, privacy/disclosure-risk, and portability/lock-in. Any failure blocks expansion and triggers a documented rollback.

• **Expand** — reuse the exact same contract + adapter pattern across domains, programs, and a

second vendor.

Five **Python** tools operationalize the gates (scorecard, contract validator, adapter conformance checker, privacy validation check, portability/lock-in assessor). They are standard-library-first, safe by default, and make no network calls.

Why it is safe. All integration and validation run in an **isolated non-production environment**, and **no real PHI is ever placed into the shared harness, twin, or AI test environment**. Vendor capabilities are integrated at the **pattern level** — version-specific details are verified against the vendor's documentation, never invented. The privacy and fidelity gates produce **evidence**; the regulatory and disclosure-risk **determinations remain with the organization's privacy, compliance, and legal functions**. The result is partner capability that is consistent, contract-defined, portable — and defensible.

Solution Overview

What this is

Digital-twin and synthetic-data capabilities frequently arrive from **external partners** — a synthetic-EHR generator, a hospital-twin simulation platform, a scenario/adversarial data service, or an AI/model framework that consumes synthetic data. Each partner solves a real problem, but integrating them *well* — consistently across programs, and without locking the organization into a single vendor — is the hard part.

This solution is the **partner/vendor integration framework** for those capabilities. It defines four things and gives you five Python tools to operationalize them:

1. **How to evaluate** a digital-twin / synthetic-data vendor against a repeatable scorecard

(fidelity, privacy posture, interoperability, portability, support, cost).

1. **The integration patterns** for plugging a vendor into your test harness and AI test

environments — data-in, data-out, and interface/contract patterns — at the architectural level, so the *shape* of the integration is standard even when the vendor changes.

1. **The data and interface contracts** — an explicit schema/interface definition the vendor

output must satisfy and your harness expects — so integrations are testable, not hand-wired.

1. **The anti-lock-in / portability criteria** — the concrete signals (open target models,

exportable artifacts, adapter isolation, exit cost) that keep a partner capability replaceable.

It is **not** a build of any specific vendor's product, and it does **not** invent vendor console internals. Vendors — Synthea-style generators, commercial synthetic-data platforms, hospital-twin engines, AI/model frameworks — are referenced by **capability and pattern**; anything version-specific is flagged "verify against your version / the vendor's documentation."

How it works

The framework standardizes the boundary between *your* environment and *any* partner capability, so the vendor becomes a **swappable component behind a stable contract** instead of a hard dependency woven through your harness.

- **Target model as the lingua franca.** Every integration is expressed against a target

clinical data model / standard — **FHIR**, **OMOP CDM**, or the source system's own schema (see the global requirements). Vendors that emit or accept the target model integrate cheaply and stay portable; vendors that only speak a proprietary shape are adapted at a single, isolated boundary and scored down on portability.

- **One adapter boundary per direction.** Vendor-specific code is isolated to a thin

data-in adapter (your reference stats / seeds → the vendor) and a **data-out adapter** (the vendor's output → your target model + harness). Your harness and AI test environments only ever see the target-model contract, never the vendor's native shape.

- **Contracts are executable.** The interface contract (schema + required fields + interface

conventions) and the adapter conformance rules are checked by the included Python tools, so a vendor either conforms or the integration fails a gate — no silent drift.

- **Portability is measured, not assumed.** A lock-in risk assessor turns the anti-lock-in

criteria into a score and a documented **exit path** for each vendor.

Target model

Integrations are authored against a defined **target clinical data model / interoperability standard** — **FHIR R4**, **OMOP CDM**, or the source system's native schema — chosen per the global requirements

(Requirement 2). The framework is model-agnostic: you declare the target model once in the integration config, and the contracts, adapters, and validation are expressed relative to it. Where a vendor's native output differs, the data-out adapter maps it to the target model and the conformance checker proves the mapping is complete.

The deployment model — Assess → Design → Generate/Build → Validate → Expand

This solution follows the house model for every Synthetic solution, applied to *partner integration*:

- **Assess** — inventory candidate vendors and the integration need; score each candidate with the vendor-evaluation scorecard against your weighted criteria. Produce `Vendor Evaluation – Candidate Set – Assess`.
- **Design** — define the target-model **interface contract** and the two adapter boundaries (data-in, data-out) for the selected vendor; specify the portability/exit requirements. Produce `Integration Contract – <Vendor Capability> – Design`.
- **Generate/Build** — implement the thin adapters and run the vendor to emit synthetic output (or execute a twin/scenario) into the **isolated non-production** environment. Produce `Vendor Output – Pilot – v1`.
- **Validate** — run the pass/fail gates: contract conformance, adapter conformance, fidelity/coverage, privacy/validation, and the portability/lock-in score. Produce `Integration Validation – Pilot – v1`. Any failed gate blocks expansion and triggers the documented rollback.
- **Expand** — once the pilot passes all gates, extend the same contract + adapter pattern to additional domains, programs, or a second vendor — reusing the exact artifact names.

Benefits

- Partner twin / synthetic-data capabilities are leveraged **consistently across programs**, not re-integrated one-off each time.
- Every integration is **contract-defined and testable**, so vendor drift is caught at a gate.
- Vendors stay **portable** — a documented exit path and adapter isolation mean a second or replacement vendor touches only the adapter, not the harness.
- **Safe by construction** — no real PHI in the shared harness/twin/AI test environment; the isolated environment and privacy gates bound residual disclosure risk.

Limitations

- The framework gives the **technical method and evidence**; it does **not** make regulatory/legal determinations. HIPAA/HITRUST de-identification determinations, IRB / privacy-office sign-off, and disclosure-risk acceptance are **deferred to the organization's privacy, compliance, and legal functions**.

• Vendor capabilities are integrated at the **pattern level**. Version-specific console steps,

API surfaces, and licensing terms must be **verified against your version and the vendor's documentation** — this solution never invents them.

• Fidelity claims are only as good as the **reference** available (global Requirement 6);

where real data cannot be used, agreed reference distributions bound what can be asserted.

• The tools are **standard-library-first** engineering aids, not a certified validation suite;

they produce evidence for human decision-makers, not approvals.

Requirements

These are the **solution-specific** prerequisites for the partner/vendor integration framework. They build on — and do **not** repeat — the six category-wide requirements in `global-requirements-synthetic.md` (a Python data environment; a target clinical data model/standard; an isolated non-production test environment; privacy/governance stakeholders and policy; AI/QA use cases and a pilot scope; read-only minimized reference access for fidelity). Confirm those six first, then verify the following.

> ***Boundary reminder.*** *Vendor capabilities are integrated at the **pattern level**; version- specific console/API details are **verified against the vendor's documentation**, never invented. Regulatory/legal determinations are **deferred** to privacy/compliance/legal. **Never** place real PHI into the shared harness, twin, or AI test environment.*

1. A candidate vendor set with obtainable evaluation artifacts

• **Description:** A named set of **candidate** digital-twin / synthetic-data vendors (or

capabilities) in scope for evaluation, and — for each — access to the artifacts needed to score them: sample output, a schema/interface description, data-handling documentation, and export/portability terms.

• **Why required:** The evaluation scorecard and the portability assessor need real inputs per

candidate; scoring on marketing claims alone is not defensible.

• **How to verify:** For each candidate you can produce (a) a small sample of its output, (b)

its documented output schema/interface, and (c) a statement of its export formats and exit terms.

• **How to implement if missing:** Request evaluation samples and documentation from each

vendor (NDA/eval agreements as needed), or use the vendor's published reference material. Record what is unavailable — missing evidence is itself a scorecard signal.

2. A declared target-model interface contract

• **Description:** The **interface contract** the integration is authored against: the target

clinical data model (FHIR / OMOP / native — from global Requirement 2), the required entities/fields in scope, and the interface conventions (file formats, identifiers, encoding, reference/foreign-key rules).

• **Why required:** The contract is the stable boundary between your harness and any vendor;

the conformance tooling checks vendor output against it. Without a declared contract there is nothing to validate portability or conformance against.

• **How to verify:** A machine-readable contract file exists (a small JSON, validated by `integration_contract_validator.py`) listing target model, entities, required fields, and interface rules.

• **How to implement if missing:** Author the contract from the target model and in-scope domains before designing the adapters (Implementation step 2).

3. Two isolated adapter boundaries (data-in, data-out)

• **Description:** A place in your architecture to host **exactly two** thin vendor-specific adapters — a **data-in** adapter (your seeds/reference stats → vendor inputs) and a **data-out** adapter (vendor output → target-model contract). Vendor-specific code lives **only** here; the harness and AI test environments consume only the target-model contract.

• **Why required:** Adapter isolation is what keeps a vendor swappable; scattering vendor SDK calls through the harness is the primary lock-in failure mode.

• **How to verify:** The harness has a single import path to each adapter and no other module references the vendor's native shape or SDK directly.

• **How to implement if missing:** Refactor to route all vendor interaction through the two adapter modules before the pilot integration.

4. Evaluation criteria with agreed weights and thresholds

• **Description:** The **weighted criteria** the organization will score vendors on (fidelity, privacy posture, interoperability, portability, support, cost, and any program-specific criteria) with agreed weights and the minimum acceptable thresholds/gates.

• **Why required:** A scorecard is only defensible if the weights and pass thresholds are agreed **before** scoring; otherwise selection looks post-hoc.

• **How to verify:** A criteria/weights file exists and is signed off by the technical and governance stakeholders (global Requirement 4).

• **How to implement if missing:** Facilitate a short weighting workshop; capture weights and thresholds in the config the scorecard tool reads.

5. A privacy/validation reference and disclosure-risk threshold for vendor output

• **Description:** The **reference** (read-only, minimized real stats or agreed reference distributions — global Requirement 6) and the **disclosure-risk threshold** the organization will hold *vendor-produced* synthetic output to, plus the quasi-identifier set to check.

• **Why required:** The privacy/validation gate must judge partner output by the same

standard as internally generated data; the threshold and quasi-identifiers are governance decisions, not tool defaults.

• **How to verify:** The quasi-identifier list, reference source, and the numeric threshold

are documented and approved.

• **How to implement if missing:** Agree them with privacy/governance before running the

privacy gate; the tool reports evidence but the **acceptance** is deferred to those stakeholders.

6. A documented exit/rollback expectation per vendor

• **Description:** For each vendor to be integrated, an agreed expectation of the **exit path**

(export formats, artifact ownership, contractual portability) and the **rollback** if a gate fails (revert to prior vendor/state, quarantine outputs).

• **Why required:** Anti-lock-in is only real if the exit is defined up front; the portability

assessor scores against it and the deployment model requires a rollback at each gate.

• **How to verify:** An exit/rollback note exists per candidate and is referenced by the

portability score.

• **How to implement if missing:** Capture the exit terms during evaluation (Requirement 1)

and the rollback during Design (Implementation step 2).

Global Requirements — Synthetic Data & AI Test-Enablement Solutions

These requirements apply to **every** solution in the **Synthetic** category (synthetic clinical data, hospital-twin simulation, and safe AI test enablement). Verify each before starting a solution's steps.

> ***Safety, vendor & regulatory boundary (applies to every Synthetic solution).*** *These solutions generate and validate* ***synthetic*** *(artificial) data and run in* ***isolated non-production*** *test environments.* ***Never place real Protected Health Information (PHI) into a shared synthetic harness, twin, or AI test environment*** *— where real data is touched at all it is read-only, minimized, and handled under the governing privacy controls.* ***Synthetic-data and digital-twin tools*** *(e.g. Synthea, and commercial platforms) and* ***AI/model frameworks*** *are integrated at the pattern/architectural level; specific vendor console internals are* ***cited or deferred*** *to that vendor's documentation, never invented.* ***Regulatory and legal interpretation*** *(HIPAA/HITRUST de-identification determinations, IRB/privacy-office sign-off, disclosure risk acceptance, and any ITAR/NERC/NRC or contract constraints) is* ***deferred to the organization's privacy, compliance, and legal functions*** *— the solutions provide the technical method and evidence, not legal determinations.*

1. A Python data/engineering environment

• **Description:** A supported **Python** environment (e.g. 3.10+) with the standard data toolchain

(data frames, statistics, and the synthetic-data / validation libraries the solution uses).

• **Why required:** The automation and reusable assets are authored in Python (generation, de-identification, fidelity/utility metrics, scenario runners).

• **How to verify:** Confirm Python + the required packages install and run in the target environment.

• **How to implement if missing:** Provision the environment and install dependencies before starting.

2. A target clinical data model / standard

• **Description:** A defined **clinical data model or interoperability standard** to generate against — e.g. **FHIR**, **OMOP CDM**, or the source system's own schema — plus the fields/domains in scope (patients, encounters, conditions, medications, labs, vitals, notes, etc.).

• **Why required:** Realistic synthetic data and a credible hospital twin require a target schema and clinically plausible relationships; validation is measured against it.

• **How to verify:** Confirm the target model/standard and the in-scope domains with the AI/QA teams.

• **How to implement if missing:** Select and document the target model before generating.

3. An isolated non-production test environment

• **Description:** A **segregated, non-production** environment (compute + storage) for the synthetic harness, hospital twin, and AI test runs — separate from production and from real-PHI systems.

• **Why required:** Safe evaluation of AI solutions before broader adoption depends on isolation; it also bounds any residual disclosure risk.

• **How to verify:** Confirm the environment is isolated, access-controlled, and not connected to production PHI stores.

• **How to implement if missing:** Stand up / obtain the isolated environment before running.

4. Privacy, governance & compliance stakeholders and policy

• **Description:** The governing **privacy, data-governance, and compliance** policies and the named stakeholders (privacy officer, security, and — where real source data or de-identification claims are involved — IRB/legal) whose requirements the work must satisfy.

• **Why required:** Regulated healthcare data constraints, de-identification acceptance, and enterprise data governance are policy decisions the technical method must align to, not defaults.

• **How to verify:** Confirm the applicable policies, disclosure-risk threshold, and approvers.

• **How to implement if missing:** Identify the policies and stakeholders before generating or releasing test data.

5. AI/QA use cases and a pilot scope

• **Description:** The **AI use cases and QA scenarios** the synthetic data / twin must support, and a **pilot scope** to validate on before broader rollout.

• **Why required:** Fidelity, coverage, and scenario design are driven by the AI use cases; every solution follows an **Assess** → **Design** → **Generate/Build** → **Validate** → **Expand** model proven on a pilot.

• **How to verify:** Confirm the target AI use cases, evaluation goals, and a named pilot.

• **How to implement if missing:** Elicit the use cases and define the pilot before building the harness.

6. Reference/real-data access for fidelity (read-only, minimized)

• **Description:** Where synthetic **fidelity/utility** must be measured against reality, **read-only, minimized** access to reference statistics or a governed real dataset (under the privacy controls in §4) — or agreed reference distributions where real data cannot be used.

• **Why required:** Fidelity and validation assets compare synthetic output to real-world distributions and relationships; without a reference the claims are unfounded.

• **How to verify:** Confirm the reference source and its access/governance terms.

• **How to implement if missing:** Agree the reference approach (governed real stats vs. published reference distributions) before validating.

*The deployment model across every Synthetic solution is the same: **Assess** → **Design** → **Generate/Build** → **Validate** → **Expand**, with explicit pass/fail (fidelity/privacy) gates and a documented rollback. Automation and reusable assets are delivered as **Python** — never real PHI is written to a shared harness, twin, or AI test environment.*

Implementation

Follow these ordered steps to evaluate a digital-twin / synthetic-data vendor, define the integration contract and adapters, run a pilot integration, and validate it against the pass/fail gates before expanding. The flow follows the house model **Assess** → **Design** → **Generate/Build** → **Validate** → **Expand**, is **pilot-first**, and names every artifact with the `<Purpose> – <Scope> – <Stage>` convention so later steps, testing, and scripts reuse the exact names.

> *Do this work in the **isolated non-production environment** (global Requirement 3). **Never** place real PHI into the shared harness/twin/AI test environment. Every writing tool defaults to **dry-run**. Version-specific vendor steps: **verify against the vendor's documentation**.*

Stage 0 — Preconditions

1. Confirm the six global requirements and the six solution-specific requirements are met.

Record gaps; a missing evaluation artifact or reference is itself an input to scoring.

1. Author the working config — three small JSON files the tools read:

• the **evaluation criteria + weights** file (Requirement 4; input to the scorecard),

• the **target-model interface contract** file (Requirement 2; validated by `integration_contract_validator.py`),

• the **integration config** (target model, adapter paths, environment = non-prod).

Name the config bundle `Integration Config – Pilot – v1`. Validate the contract file up front with `integration_contract_validator.py --contract ...` before proceeding.

Stage 1 — Assess (evaluate candidate vendors)

1. For each candidate in the vendor set (Requirement 1), gather the evaluation inputs: a sample of output, its schema/interface doc, data-handling doc, and export/exit terms. Store them read-only under the candidate's evaluation folder.

1. Score every candidate with the scorecard tool (`vendor_evaluation_scorecard.py`) against the agreed weighted criteria. Produce `Vendor Evaluation – Candidate Set – Assess` (a ranked scorecard with per-criterion scores, the portability sub-score, and any missing- evidence flags).

1. **Gate A — Selection.** Review the ranked scorecard with technical + governance stakeholders. Select the vendor(s) that clear the agreed thresholds (Requirement 4). Failing all thresholds → do not proceed; revisit the candidate set. Record the decision on the `Vendor Evaluation – Candidate Set – Assess` artifact.

Stage 2 — Design (contract + adapters + exit)

1. Author the **interface contract** for the selected vendor against the target model (FHIR / OMOP / native): declare entities, required fields, identifier/reference rules, and interface conventions. Validate the contract file itself with the contract validator (`integration_contract_validator.py --contract ...`). Produce `Integration Contract – <Vendor Capability> – Design`.

1. Specify the **two adapter boundaries**:

• **data-in adapter** — how your seeds / reference stats map to the vendor's inputs,

• **data-out adapter** — how the vendor's native output maps to the target-model contract.

Keep all vendor-specific code in these two modules only (Requirement 3).

1. Capture the **exit path and rollback** for this vendor (export formats, artifact ownership, revert plan). Produce `Portability & Exit Plan – <Vendor Capability> – Design`. This feeds the portability assessor and the Validate gates.

Stage 3 — Generate/Build (run the pilot integration)

1. Implement the two thin adapters per the Design. Do not reference the vendor's native shape anywhere outside them.

1. Run the vendor capability to produce pilot output in the isolated environment — either:

• **synthetic generation** (emit a synthetic dataset against the target model), or

• **twin/scenario execution** (run the hospital-twin/scenario and capture its output).

Route the result through the **data-out adapter** into the target-model contract. Produce `Vendor Output – Pilot – v1` (target-model-shaped, in the isolated environment only).

Stage 4 — Validate (the pass/fail gates)

Run each gate on `Vendor Output – Pilot – v1`. Record results in `Integration Validation – Pilot – v1`. Any failed gate **blocks expansion** and triggers the rollback from step 8.

1. **Gate B — Contract conformance.** Run `integration_contract_validator.py` with the

`Integration Contract – <Vendor Capability> – Design` against the vendor output. **Pass:** all required entities/fields present, identifier/reference rules satisfied, no contract violations. **Fail:** any missing required field or broken reference.

1. **Gate C — Adapter conformance.** Run `adapter_conformance_checker.py` on the data-in and

data-out adapters with paired input/output fixtures. **Pass:** round-trip is lossless for in-scope fields and no vendor-native shape leaks past the adapter. **Fail:** field loss, type drift, or a native-shape leak.

1. **Gate D — Fidelity/coverage.** Compare the output to the reference (global Requirement 6)

as a documented procedure (see Testing, Test 3): in-scope column distributions vs. the read-only reference and the required scenario/domain coverage matrix. **Pass:** in-scope distributions and required scenario coverage meet the agreed thresholds. **Fail:** below threshold on a required metric or coverage gap.

1. **Gate E — Privacy/validation.** Run `privacy_validation_check.py` with the agreed quasi-

identifiers and disclosure-risk threshold (Requirement 5). **Pass:** no exact real-record matches, uniqueness/k-anonymity signals within the agreed threshold. **Fail:** any signal over threshold. *(Evidence only — acceptance is deferred to privacy/compliance.)*

1. **Gate F — Portability/lock-in.** Run `portability_lockin_assessor.py` against the

`Portability & Exit Plan` and the contract. **Pass:** portability score at/above the agreed minimum, exit path documented and exercised (a test export succeeds). **Fail:** score below minimum or no working export.

1. **Gate summary.** All of Gates B–F must pass to proceed. Attach the tool outputs to

`Integration Validation – Pilot – v1` and obtain the governance sign-off recorded there.

Stage 5 — Expand

1. With the pilot passing all gates, extend the **same** contract + adapter pattern to the

next domain, program, or a second vendor. Reuse the exact artifact names, incrementing the scope/stage (e.g. `Vendor Output – Cardiology – v1`, `Integration Contract – <Second Vendor> – Design`). Re-run Gates B–F for each expansion.

1. Maintain a **vendor register** linking each integrated capability to its contract,

adapters, portability/exit plan, and latest validation — so partner capabilities stay consistently governed and portable across programs.

Testing & Validation

These procedures are the pass/fail gates for a partner integration. Run them on `Vendor Output – Pilot – v1` (and on each expansion) and record results in `Integration Validation – Pilot – v1`. Each gate has an explicit **expected pass/fail**. Privacy and fidelity gates produce **evidence**; the regulatory/disclosure **acceptance** is deferred to the organization's privacy, compliance, and legal functions.

> *All validation runs in the **isolated non-production environment**. **Never** load real PHI into the shared harness/twin/AI test environment. The tools are safe-by-default (read-only or dry-run) and produce evidence, not approvals.*

Test 1 — Contract conformance (Gate B)

- **Purpose:** Confirm vendor output satisfies the declared target-model interface contract.
- **Tool:** `integration_contract_validator.py --contract "Integration Contract – <Vendor Capability> – Design" --data <Vendor Output – Pilot – v1>`
- **Procedure:** Validate required entities, required fields, types, and identifier/reference (foreign-key) rules against the contract.
- **Expected — PASS:** all required entities/fields present and typed correctly; every reference resolves; zero contract violations.
- **Expected — FAIL:** any missing required field, wrong type, or unresolved reference. Fix in the data-out adapter, re-run.

Test 2 — Adapter conformance / round-trip (Gate C)

- **Purpose:** Confirm the data-in and data-out adapters are lossless for in-scope fields and that no vendor-native shape leaks past the adapter boundary.
- **Tool:** `adapter_conformance_checker.py --direction both --fixtures <paired fixtures>`
- **Procedure:** Feed known fixtures through data-in → (vendor) → data-out and compare against the expected target-model records; scan the output for vendor-native keys.
- **Expected — PASS:** in-scope fields round-trip without loss or type drift; no vendor-native keys present in the harness-facing output.
- **Expected — FAIL:** any dropped/renamed in-scope field, type drift, or a leaked native key.

Test 3 — Fidelity & coverage (Gate D)

- **Purpose:** Confirm the vendor output is statistically credible against the reference and covers the required scenarios/domains.

• **Procedure (documented comparison):** Compare in-scope column distributions of `Vendor Output – Pilot – v1` to the read-only reference (global Requirement 6) — e.g. category frequencies and numeric summary statistics / divergence — and complete the required scenario/domain **coverage matrix**. Record the metrics and the agreed tolerances alongside the result. (This gate is procedural against your reference rather than a bundled tool, since the reference and tolerances are organization-specific.)

• **Expected — PASS:** every required distribution within the agreed tolerance and all required scenarios/domains covered.

• **Expected — FAIL:** any required metric outside tolerance or a coverage gap. Regenerate or adjust scope; re-run.

Test 4 — Privacy / disclosure-risk validation (Gate E)

• **Purpose:** Confirm vendor-produced synthetic output does not reproduce real records and its disclosure-risk signals are within the agreed threshold.

• **Tool:** `privacy_validation_check.py --data <Vendor Output – Pilot – v1> --reference <read-only reference> --quasi-identifiers <QI list> --k <agreed k>`

• **Procedure:** Check for exact matches against the read-only reference, compute uniqueness / k-anonymity on the agreed quasi-identifiers, and flag any record below k.

• **Expected — PASS:** zero exact real-record matches; uniqueness/k-anonymity signals within the agreed threshold.

• **Expected — FAIL:** any exact match or any signal over threshold. Quarantine the output; escalate to privacy/governance. **Acceptance of residual risk is deferred** to those stakeholders — a PASS here is evidence, not a compliance determination.

Test 5 — Portability / lock-in (Gate F)

• **Purpose:** Confirm the vendor remains replaceable — adapter isolation holds, the exit path works, and the portability score meets the agreed minimum.

• **Tool:** `portability_lockin_assessor.py --plan "Portability & Exit Plan – <Vendor Capability> – Design" --contract "Integration Contract – <Vendor Capability> – Design"`

• **Procedure:** Score the anti-lock-in criteria (open target model, exportable artifacts, adapter isolation, documented exit, exit cost) and exercise a **test export** of the vendor output to a portable format.

• **Expected — PASS:** portability score at/above the agreed minimum **and** the test export succeeds (output re-loads without vendor tooling).

• **Expected — FAIL:** score below minimum or the export cannot be produced/re-loaded. Improve

adapter isolation / negotiate exit terms before expanding.

Gate summary & rollback

- **All of Gates B–F must PASS** before expansion (Implementation Stage 5).
- A failed gate **blocks expansion** and triggers the documented **rollback** (revert to the prior vendor/state, quarantine `Vendor Output – Pilot – v1`).
- Record every gate result, the tool output, and the governance sign-off in `Integration Validation – Pilot – v1`.

Troubleshooting

Common failure modes when evaluating and integrating partner digital-twin / synthetic-data capabilities, and how to resolve them. All work stays in the **isolated non-production environment**; **never** load real PHI into the shared harness/twin/AI test environment.

Gate A — no candidate clears the selection threshold

- **Symptom:** `vendor_evaluation_scorecard.py` reports every candidate BELOW-THRESHOLD.
- **Likely causes:** thresholds/weights set too strict; or several candidates scored 0 on criteria only because **evidence was missing** (shown in the `missing` column).
- **Resolution:** First close the *missing-evidence* gaps — request the sample output, schema, or export terms and re-score (missing evidence is scored as 0 by design). If genuine scores are still low, revisit the candidate set with governance rather than lowering the agreed threshold post-hoc.

Scorecard weights don't sum to 1.0

- **Symptom:** `NOTE: weights sum to X; normalizing to 1.0` on stderr.
- **Cause:** the `--criteria` file weights don't total 1.0.
- **Resolution:** The tool normalizes automatically, but fix the criteria file so the agreed weights are explicit and auditable — selection defensibility depends on the agreed weights.

Contract file invalid (before any data)

- **Symptom:** `CONTRACT INVALID` with messages like "type not in …" or "reference to unknown entity".
- **Causes:** a field type outside the allowed set (`string, integer, number, boolean, date, datetime`); a reference pointing at an entity not declared in the contract.
- **Resolution:** Correct the interface-contract JSON. Every referenced entity must be declared,

and every field must use an allowed type. Re-run `integration_contract_validator.py --contract ...` until you get `CONTRACT OK`.

Gate B FAIL — missing fields or broken references

- **Symptom:** `GATE B: FAIL` listing missing required fields, wrong types, or …`has no matching <Entity>.<key>`.
- **Cause:** the vendor's native output isn't fully mapped to the target model, or referential integrity is broken in the mapping.
- **Resolution:** Fix the **data-out adapter** (not the harness): map the missing/renamed fields, coerce types, and ensure foreign keys resolve. Re-run against `Vendor Output – Pilot – v1`. Do **not** relax the contract to make output pass — that erodes portability.

Gate C FAIL — vendor-native shape leaking into the harness

- **Symptom:** `adapter_conformance_checker.py` reports `vendor-native leak: [...]` or lost/drifted in-scope fields.
- **Cause:** the adapter is passing vendor-native keys straight through, or dropping/renaming in-scope fields — the classic lock-in defect.
- **Resolution:** Make the data-out adapter emit **only** target-model keys; strip vendor-native keys (and anything prefixed to look native). Restore any lost in-scope fields. Keep the adapter thin — no business logic. Re-run until `GATE C: PASS`.

Gate D — fidelity below tolerance or coverage gaps

- **Symptom:** an in-scope distribution is outside the agreed tolerance, or a required scenario/domain isn't present.
- **Causes:** vendor configuration/seed doesn't match the reference population; scenario suite under-specified; or the reference itself is unrepresentative.
- **Resolution:** Re-configure the vendor generation/scenario parameters toward the reference, or expand the scenario suite for the missing coverage. If the **reference** is the problem, agree a better reference or reference distributions with governance (global Requirement 6) before asserting fidelity.

Gate E FAIL — exact matches or below-k records

- **Symptom:** `privacy_validation_check.py` reports exact QI-matches against the reference or QI groups below k.
- **Causes:** the vendor output memorized/reproduced real records; or quasi-identifier combinations are too granular (e.g. full ZIP + exact DOB).

• **Resolution:** Quarantine the output. Reduce granularity (generalize QIs), regenerate, and re-run. Escalate any exact match to privacy/governance. Remember: a PASS is **evidence** — the disclosure-risk **acceptance is deferred** to privacy/compliance/legal, never inferred from a green result.

• **Also:** ensure `--reference` points at a **read-only, minimized, governed** copy — never load real PHI into the shared harness to run this check.

Gate F FAIL — low portability score or export won't re-load

• **Symptom:** `portability_lockin_assessor.py` reports a score below the minimum, a MISSING criterion, or `test export … FAILED to re-load`.

• **Causes:** vendor code not isolated to the two adapters; output only available in a proprietary format; no documented exit; or the exit was never actually exercised.

• **Resolution:** Improve **adapter isolation** (Gate C), obtain an open export path, and document the exit/rollback in `Portability & Exit Plan – <Vendor Capability> – Design`. Then exercise a real export with `--data` + `--export-out` and confirm it re-loads **without vendor tooling**. A score that has never produced a working export does not pass.

A gate passes but the vendor still feels "sticky" across programs

• **Symptom:** integrations work but each new program re-wires the vendor differently.

• **Cause:** the contract + adapter pattern isn't being reused; no central register.

• **Resolution:** Reuse the exact `Integration Contract` + adapter pattern on every expansion and maintain the **vendor register** linking each capability to its contract, adapters, exit plan, and latest validation (Implementation Stage 5). Consistency across programs is a governance practice, not an automatic outcome.

Version drift after a vendor upgrade

• **Symptom:** a previously passing integration starts failing Gate B/C after the vendor updates.

• **Cause:** the vendor changed its native output shape or export behavior.

• **Resolution:** Vendor capabilities are integrated at the **pattern level** — re-verify the changed behavior against **the vendor's documentation for that version**, update the adapter only, and re-run Gates B–F. Record the verified version. Do not hard-code assumptions about a vendor's internals.

Best Practices

Durable practices for integrating partner digital-twin / synthetic-data capabilities so they are leveraged consistently and stay portable.

Treat the target model as the contract, not the vendor

• Author every integration against the declared **target model** (FHIR / OMOP / native), never against a vendor's native shape. Your harness and AI test environments should only ever see the target-model contract.

• Keep the interface contract **machine-readable and versioned**. When it changes, bump the contract version and re-run Gate B on every integrated vendor.

Isolate vendor code to exactly two adapters

• All vendor-specific code lives in the **data-in** and **data-out** adapters — nowhere else. If a vendor SDK call or native key appears in the harness, that is a lock-in defect; fix it before it spreads.

• Keep adapters **thin**: map shapes, don't add business logic. Logic in an adapter is logic you re-implement when you swap vendors.

Score before you select, and keep the evidence

• Agree the **weighted criteria and thresholds before** scoring (Requirement 4) so selection is defensible, not post-hoc. Treat *missing evaluation evidence* as a scoring signal, not a neutral gap.

• Re-score periodically. A vendor's fidelity, privacy posture, or portability can drift across versions — re-run the scorecard at major upgrades.

Design the exit on day one

• Capture the **exit path and rollback per vendor during Design**, not at renewal time. A vendor you cannot leave cheaply is a vendor that sets your price.

• **Exercise the exit** — a portability score that has never produced a working export is aspirational. The portability gate requires a real test export that re-loads without vendor tooling.

Hold partner output to your own privacy bar

• Judge vendor-produced synthetic data by the **same disclosure-risk threshold and quasi-identifier checks** as internally generated data. "It's synthetic, from a vendor" is not a privacy control.

• Keep any real reference **read-only, minimized, and out of the shared harness**. The privacy gate produces **evidence**; **defer the acceptance** to privacy/compliance/legal — never let a green tool output stand in for a de-identification determination.

Keep the safety and boundary discipline

• **Never** place real PHI into the shared harness, twin, or AI test environment. Do all integration and validation in the **isolated non-production environment**.

• Reference vendor capabilities at the **pattern level**; for anything version-specific, verify

against the vendor's documentation rather than assuming — and record the version you verified.

Govern the portfolio, not just the pilot

- Maintain a **vendor register** linking each capability to its contract, adapters, portability/

exit plan, and latest validation. This is what makes partner capabilities *consistent across programs* instead of a set of one-off integrations.

- Re-run Gates B–F on every expansion and every vendor upgrade — conformance and portability

are properties to keep, not achieve once.

Deliverables Summary

What the buyer gets — a self-guided framework for evaluating, integrating, and keeping portable any partner digital-twin / synthetic-data capability, plus five Python tools that operationalize the gates.

Documentation

- **Solution Overview** — the integration model for external twin/synthetic-data capabilities:

evaluation, integration patterns (data-in / data-out / interface contract), the target-model contract, portability, and the Assess → Design → Generate/Build → Validate → Expand model.

- **Requirements Guide** — the six solution-specific prerequisites (candidate vendor set,

target-model contract, isolated adapter boundaries, weighted evaluation criteria, privacy reference/threshold, exit/rollback), each with how to *verify* and *implement*, on top of the six category-wide requirements.

- **Implementation Guide** — the exact ordered steps: evaluate candidates, design the contract

and two adapters, run a pilot integration, and pass Gates A–F before expanding. Every artifact named with the `<Purpose> – <Scope> – <Stage>` convention.

- **Testing & Validation** — the five pass/fail gates (contract, adapter, fidelity/coverage,

privacy, portability) with expected results and rollback.

- **Best Practices** — durable practices for low-lock-in, consistently governed partner

integration.

- **Troubleshooting** — common failure modes and resolutions.

- **Executive Summary** — outcome, approach, and why it is safe.

The 5 Python tools

1. `vendor_evaluation_scorecard.py` — scores candidate vendors against agreed weighted

criteria (fidelity, privacy, interoperability, portability, support, cost), ranks them, and flags missing evaluation evidence. Produces the `Vendor Evaluation – Candidate Set – Assess` scorecard.

1. `integration_contract_validator.py` — validates a target-model **interface contract**

file, and validates vendor output against it (required entities/fields, types, identifier/reference rules). The Gate B tool.

1. `adapter_conformance_checker.py` — checks the **data-in / data-out** adapters for

lossless round-trip of in-scope fields and detects vendor-native shape leaks past the adapter boundary. The Gate C tool.

1. `privacy_validation_check.py` — checks vendor-produced synthetic output for exact

real-record matches and computes uniqueness / k-anonymity signals on the agreed quasi-identifiers against a read-only reference. The Gate E tool (evidence only; acceptance deferred to privacy/compliance).

1. `portability_lockin_assessor.py` — scores the anti-lock-in / portability criteria (open

target model, exportable artifacts, adapter isolation, documented + exercised exit, exit cost) and produces a lock-in risk rating with a documented exit path. The Gate F tool.

(Gate D — fidelity/coverage — is run as a documented comparison against the reference, per Testing; the criteria/weights, contract, and integration config that the tools above read are plain JSON files you author during Implementation Stage 0.)

All tools are **standard-library-first**, run on **Python 3.10+**, are **safe by default** (read-only or `--dry-run` for anything that writes), make **no network calls**, and never require or write real PHI. They produce **evidence** for human decision-makers, not approvals.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

adapter_conformance_checker.py

integration_contract_validator.py

portability_lockin_assessor.py

privacy_validation_check.py

vendor_evaluation_scorecard.py